

Disclaimer: Dieses OER-Dokument stammt aus dem Projekt [proTirone](#), das über GitHub [freie Unterrichtsmaterialien](#) samt [Quellcode](#) offeriert. proTirone-Materialien werden unter den Bedingungen der [CC-BY-4.0-Lizenz](#) so angeboten, wie sie sind, ohne Zusage bestimmter Eigenschaften und ohne Gewährleistung (§5). Dafür dürfen sie – bei angemessener Namensnennung (§3) – für beliebige (auch kommerzielle) Zwecke verändert und weitergegeben werden (§2).

LFCX:RAID:Datensicherung mit / durch RAID

ÜBUNG LFCX:RAID:01

Gehen Sie von folgenden Fakten aus:

- Es gab eine Zeit, da waren gute Festplatten richtig teuer. Richtig, richtig teuer. Auch wenn sie längst nicht so viel Speicherkapazität boten, wie heutige. Denn eine Haupteigenschaft der guten Festplatten war ihre Ausfallsicherheit.
- Und es gab zu diesen Zeiten auch kostengünstige Festplatten. Sie waren so günstig (im Vergleich), dass das Ausfallrisiko sie nicht vom Markt verdrängt hat.

Also sind findige Menschen auf die Idee gekommen, einen ausfallsicheren Speicherplatzverbund mit mehreren günstigen, aber unsicheren Platten zu konzipieren. Darauf bezieht sich Ihre Aufgabe:

- ☐ Teilen Sie sich in 4 (3) Gruppen auf.
- ☐ Skizzieren Sie eine Idee, wie eine sichere Speichertechnikethik mit unsicheren Platten verwirklicht werden könnte.
- ☐ Gehen Sie ans Whiteboard:
 - ☐ Wenn Ihre Idee dort schon notiert ist, finden sie eine andere.
 - ☐ Wenn nicht, tragen Sie Ihre Idee als Teaser dort ein.
- ☐ Konzipieren Sie Ihre Idee im Detail.
- ☐ Beschreiben Sie, wie man den Umfang der Ressourcen berechnet, die für Ihre Idee nötig sind. (Von konkreten Produkten können Sie abstrahieren)
- ☐ Formulieren / Konzipieren Sie eine kurze(!) Präsentation, anhand derer Sie Ihren Mitschülerinnen den Kern und den Witz Ihrer Idee erläutern können.

Für die Entwicklung Ihres Konzeptes und Ihrer Präsentation haben Sie 50 Minuten. Danach präsentiert jede Gruppe Ihren Ansatz.

Anknüpfend an die Ergebnisse der Schülerinnen, bringt die Lehrerin folgende Zusatzinfos ein:

1. RAID [→ ZP:Sheet:2]

steht für:

- **‘Redundant Array of Independent Disks’** (heute)
- *Redundant Array of Inexpensive Disks* (früher)

Grund für Umbenennung:

- heute sind Festplatten kein (wesentlicher) Kostenfaktor mehr,
- wohl aber die Lese- und Schreiboperationen (die den Datendurchsatz verlängern)

Zweck: Ein Verbund von Festplatten soll die *Mean Time Between Failures (MTBF)* erhöhen

RAID 0

- meint **Striping**
- zwei Platten werden
 - “in zusammenhängende Blöcke gleicher Größe aufgeteilt”
 - “im Reißverschlussverfahren zu einer [virtuellen] großen Festplatte angeordnet”.
- dient der Lese Beschleunigung ohne Redundanz
 - Erhöht nur Datendurchsatz
 - Fällt eine Platte aus, ist das Gesamtsystem zerstört
 - Eigentlich keine Redundanz, also kein Raid

Bei n Platten der Größe S

- $\text{Nutzdaten} = n \cdot S$

RAID 1

- meint **Mirroring** (*Spiegelung*)
- Zwei (oder mehr) Platten werden
 - “in zusammenhängende Blöcke gleicher Größe aufgeteilt”
 - jeweils ‘gleichzeitig’ mit denselben Daten beschrieben.
- bietet volle Redundanz
 - fällt eine Platte aus, kann die andere einspringen bzw. die Ersatzplatte restaurieren.
 - Erhöht nur Lesedurchsatz

- *Subtypen:*
 - **Mirroring** im engeren Sinne
 - * beide Platten nutzen denselben Controller. (1 Schreibbefehl, zweimal sequentiell ausgeführt)
 - * Nachteil (I): Controller ist Single Point of Failure
 - * Nachteil (II): Schreibprozess verlangsamt
 - **Duplexing:** jede Platte ein eigener Controller
 - * Vorteil (I): fällt Controller aus, bedient der zweite den Schreib und Leseprozess auf der anderen Platten
 - * Vorteil (II): Schreiboperation schneller

Bei n Platten der Größe S

- **Nutzdaten** = S

RAID 4

- meint **Block-Level Striping mit gesonderter Paritätsinformation**
- n Festplatten
 - bilden einen Verbund
 - aus den nächste n-1 Datenpaketen wird per XOR ein n. Datenpaket generiert
 - die n-1 Datenpakete werden im Reißverschlussverfahren (Striping) in dieselben Sektoren/-Blöcke der Platten 1 - n-1 geschrieben,
 - die Paritätsinformation wird in denselben Sektor/Block der Platte n geschrieben

Paritätsinformationen / -daten eines Paritätsblocks

- werden “durch XOR-Verknüpfung der Daten aller Datenblöcke der Gruppe” berechnet,
- machen Daten restaurierbar: Paritätszahl xor erhaltener Block = Inhalt von zerstörtem Block
- Vorteil:
 - 1 Platte ist im Betrieb austauschbar
 - alle Nutzdatenplatten stehen für Leseoperationen zur Verfügung
- Nachteil:
 - Je mehr Kapazität der *virtuellen* Platte, je teurer der reale Aufwand
 - Paritätsplatte ist der Single Point of Failure

Bei n Platten der Größe S

- $\text{Nutzdaten} = (n-1) * S$
- $\text{Paritätsdaten} = S$

RAID 5

- meint **Block-Level Striping mit verteilter Paritätsinformation**
- n Festplatten
 - bilden einen Verbund
 - aus den nächste n-1 Datenpaketen wird per XOR ein n. Datenpaket generiert
 - die n Datenpakete werden Nutzungsdaten im Reißverschlussverfahren (Striping) geschrieben,

Paritätsinformationen / -daten eines Paritätsblocks

- werden “durch XOR-Verknüpfung der Daten aller Datenblöcke der Gruppe” berechnet,
- machen Daten restaurierbar: Paritätszahl xor erhaltener Block = Inhalt von zerstörtem Block [*→ ZP:Sheet:3*]
- Vorteil:
 - 1 Platte ist im Betrieb austauschbar
 - alle Platten stehen für Leseoperationen zur Verfügung
- Nachteil: Je mehr Kapazität der *virtuellen* Platte, je teurer der reale Aufwand

Bei n Platten der Größe S

- $\text{Nutzdaten} = (n-1) * S$
- $\text{Paritätsdaten} = S$

RAID 6

- meint **Block-Level Striping mit verteilter doppelten Paritätsinformation**
- n Festplatten
 - bilden einen Verbund
 - aus den nächste n-2 Datenpaketen wird per XOR ein n-1. und - verschoben - ein n.1 Datenpaket generiert
 - die n Datenpakete werden Nutzungsdaten im Reißverschlussverfahren (Striping) geschrieben,
- Vorteil:

- 2 Platten sind im Betrieb austauschbar
- alle Platten stehen für Leseoperationen zur Verfügung
- Nachteil: Je mehr Kapazität der *virtuellen* Platte, je teurer der reale Aufwand

Bei n Platten der Größe S

- $\text{Nutzdaten} = (n-2) * S$
- $\text{Paritätsdaten} = 2S$

Sonderformen

- **RAID 4** :- wie *Raid 5*, nur dass alle Paritätsinfos auf dieselbe Platte geschrieben werden. (Single Point of Failure)
- **RAID 10** :- Vereinigung von RAID-0 (Striping) und RAID-1 (Mirroring)
 - Daten werden über 2 je gespiegelte Paare gestriped. Verlangt also mindestens 4 Platten.

(vgl <https://de.wikipedia.org/wiki/RAID>) [→ *ZP:Sheet:2*]

2. Paritätsinformationen [→ *ZP:Sheet:3*]

- sollen dazu dienen, aus den erhalten gebliebenen Platten und eben diesen Paritätsinformationen den Inhalt einer zerstörten Platten zu rekonstruieren,
- arbeiten im Kern mit der bitweisen XOR-Verknüpfung: Jede beteiligte Festplatte wird als Folge von Bytes / Blocks verstanden, sodass
 - a) die Bytes / Blocks an der jeweils selben Position herausgesucht,
 - b) miteinander bitweise xor-verküpft werden und
 - c) das Ergebnis diese Berechnung an derselben Position abgespeichert wird

		char(Byte N)	dec(Byte N)	bin(Byte N)
	Platte A	'A'	65	0100 0001
xor	Platte B	'y'	121	0111 1001
→	Platte C	'8'	56	0011 1000

		char(Byte N)	dec(Byte N)	bin(Byte N)
	Platte C	'8'	56	0011 1000
xor	Platte B	'y'	121	0111 1001
→	Platte A	'A'	65	0100 0001

		char(Byte N)	dec(Byte N)	bin(Byte N)
	Platte C	'8'	56	0011 1000
xor	Platte A	'A'	65	0100 0001
→	Platte B	'y'	121	0111 1001

Es ergibt sich also:

- `xor(xor('A', 'y'), 'A') == 'y'`
- `xor(xor('A', 'y'), 'y') == 'A'`

Hinweis:

Dies gilt auch für in der Reihe verknüpfte Operanden, sofern die Reihenfolge bei der Rekonstruktion nicht geändert wird.

ÜBUNG LFCX:RAID:01

- ☐ A) Konzipieren Sie einen mathematischen Beweis dafür, dass die Wiederherstellbarkeit für beliebige lange Binärzahlen gilt.
- ☐ B) Konzipieren Sie einen mathematischen Beweis dafür, dass die Wiederherstellbarkeit für beliebige viele XOR-Verkettungen beliebig langer Binärzahlen gilt.

[→ ZP:Sheet:4]

Hintergrund: In der Informatik/Mathematik gibt es 3 (bzw. 4) Arten der Beweisformen

- Ein **Deduktionsbeweis**:
 - Gegeben sei eine gültige Regel der Form **für alle x gilt: wenn $p(x) \rightarrow q(x)$**
 - Wenn außerdem gegeben ist ein Faktum der Form

- * $p(A)$, dann folgt daraus - gemäß Modus Ponens - $q(A)$ (weil die Wahrheit der Implikation für alle x gilt: wenn $p(x) \rightarrow q(x)$ es nicht zulässt, dass nicht ($q(A)$))
- * $\text{non } q(A)$ dann folgt daraus - gemäß Modus Tollens - $\text{non } p(A)$ (weil die Wahrheit der Implikation für alle x gilt: wenn $p(x) \rightarrow q(x)$ es nicht zulässt, dass nicht($p(A)$))

Beweis über Wahrheitstabellen:

$p(A)$	$q(A)$	$p(A) \rightarrow q(A)$
w	w	w
w	f	f
f	w	w
f	f	w

Also:

- wenn $p(A) = w$ und $p(A) \rightarrow q(A) = w$, muss $q(A) \neq f$ (Modus Ponens)
- wenn $q(A) = f$ und $p(A) \rightarrow q(A) = w$, muss $p(A) = f$ (Modus Tollens)

Aber: Deduktion als Beweis hier nicht anwendbar, da nicht alle unendlich vielen Einzelfälle auflistbar

- Ein **Induktionsbeweis**:
 - Gegeben sei eine erfüllte Startbedingung
 - * Reproduzierbarkeit zweier 1-Bit-Zahlen per XOR-Restaurierung gelingt. Beweis mit XOR-Definition.
 - Induktionsvoraussetzung: Für zwei beliebige n -Bit-Zahlen $nb1$ und $nb2$ sei angenommen, deren XOR-Reproduzierbarkeit sei bereits nachgewiesen.
 - Gesetzt, die beiden Zahlen $nb1+$ und $nb2+$ seien je aus $nb1$ bzw. $nb2$ gewonnen, in dem diese um 1 Bit verlängert seien.
 - Dann gilt:
 - * Werden die beiden neuen Bits XOR-Verknüpft sind sie für sich XOR-reproduzierbar. (Folgt aus XOR-Definition)
 - * Die Zahlen $nb1$ und $nb2$ sind XOR-reproduzierbar. Die beiden neuen Bits werden bei der Reproduktion von $nb1$ bzw. $nb2$ nicht beeinflusst. (Kein Bit-Übertrag. Folgt aus XOR-Definition)
 - Also $nb1+$ und $nb2+$ sind auch XOR-reproduzierbar

- Also unendlich viele

Aber: Es müsste noch genau formuliert werden, dass und wie die XOR-reproduzierbarkeit auf die XOR-Definition zurück geführt werden kann.

- Ein **Widerspruchsbeweis:**

Klassisches Beispiel: Zu beweisen ist, dass es unendlich viele Primzahlen gibt.

- (1) Angenommen, es gäbe nur endlich viele Primzahlen.
- (2) Dann müsste es eine höchste Primzahl geben. Nennen wir die hpz
- (3) hpz ist nur durch 1 und hpz teilbar. (Folgt aus Primzahl)
- (4) Sei $twohpz = 2 * hpz$.
 - (5) $twohpz$ ist durch 1, durch hpz und durch 2 teilbar
- (6) Sei $twohpz_pl_1 = twohpz + 1$.
 - (7) $twohpz_pl_1$ ist durch 1, durch hpz aber NICHT durch 2 (oder sonst wie teilbar.
- (8) Also: $twohpz_pl_1$ ist eine Primzahl und größer als hpz
- (9) hpz sollte aber die größte Primzahl sein. Widerspruch zu (1)

Also gilt **non** (1)

Lösung XOR-Reproduzierbarkeit:

- **Induktionsbeweis:**

- Für **A:** Induktionsbeweis über die Anzahl der Binärstellen der beiden beteiligten Ausgangszahlen (wie o.a.).
- Für **B:** Induktionsbeweis über die Anzahl der XOR-Verknüpfungen (unter Rückgriff auf Beweis zu A) analog.

- **Beweis durch Widerspruch:**

- Für **A:** Zu beweisen ist, XOR-Reproduzierbarkeit für Paare beliebig langer gleichlanger Bitzahlen gilt.
 - * (1) Gesetzt, nicht alle Paare beliebig langer gleichlanger Bitzahlen sind XOR-reproduzierbar.
 - * (2) Dann müsste es ein Paar gleichlanger Bitzahlen $\langle bz1, bz2 \rangle$ geben, bei denen an mindestens einer beliebigen ersten Bit-Position das in der Prüfwahl gesetzte/nicht-gesetzte Bit nicht den Regeln der XOR-Verknüpfung folgt. Sei n die Position.

- * (3) Alle Bits in der Prüfzahl an kleiner Position als n folgen dem XOR-Regeln.
- * (4) Die XOR-Regeln sehen keinen Überlauf vor (die Bitsetzung an Position m beeinflusst nicht die Bitsetzen an Position $m+1$)
- * (5) Also folgen die Bitsetzung an Position n auch den Regeln der XOR-Regeln.
- * (6) Also ist n nicht die Position, wo zum ersten Mal die XOR-Setzungsregeln gebrochen werden. Widerspruch!
- * Also gilt **non** (1)
- Für **B**: Induktionsbeweis über die Anzahl der XOR-Verknüpfungen (unter Rückgriff auf Beweis zu A) analog.

Zweck der Beweistechniken für Fachinformatiker:

Beweis der Softwarekorrektheit: [*→ ZP:Sheet:5*]
